Reviewer Pack — Minimal Rank of Geometric Measure Theory over Max-Cut Relaxa…

Entry 6ade8b9067eb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 11:22:11 UTC

Minimal Rank of Geometric Measure Theory over Max-Cut Relaxations

Entry ID: `6ade8b9067eb` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 11:22:11 UTC

1. Conjecture

Field A (mathematical branch): Geometric Measure Theory **Field B** (complexity object): Complexity Theory: Sum-of-squares Complexity (for Max-CUT)

Statement:

> For any degree-d sum-of-squares polynomial p representing a max-CUT relaxation, the minimal rank of its associated moment matrix M_p is bounded by O(d * log^2(n)) where n is the number of variables. For all instances with property P, property Q holds: if M_p has a rank less than O(d * log^2(n)), then p provides an approximation ratio worse than 0.878 for max-CUT.

Rationale (proposer's reasoning):

> Geometric Measure Theory offers powerful tools to study the geometry of high-dimensional spaces, which might shed light on the structure of moment matrices associated with max-CUT relaxations. If the conjecture holds, it would suggest a direct relationship between the geometric properties of the moment matrix and the approximation ratio of the sum-of-squares polynomial, potentially leading to new algorithmic insights.

Taxonomy category: `SOS_DEGREE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `3aaa1e2cc8ce5b3e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a given max-CUT instance with n ≤ 40 variables and degree-d sum-of-squares polynomial p, if the rank of its moment matrix M_p is less than O(d * log^2(n)), then the approximation ratio of p for max-CUT must be worse than 0.878.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `geometric measure theory AND sum-of-squares complexity AND max-cut relaxation` - `moment matrix rank AND geometric measure theory AND Max-CUT approximation` - `sum-of-squares polynomial AND minimal rank AND geometric measure theory Max-CUT`

Top relevant hits considered: - [http://arxiv.org/abs/1809.04266v1] A Singular Integral Measure for $C^{1,1}$ and C^1 Boundaries - [http://arxiv.org/abs/1911.08704v2] Node Max-Cut and Computing Equilibria in Linear Weighted Congestion Games - [http://arxiv.org/abs/2403.13102v1] Geometric methods in quantum information and entanglement variational principle - [http://arxiv.org/abs/2109.02238v1] Optimal solutions and ranks in the max-cut SDP - [http://arxiv.org/abs/2105.12016v4] Waring ranks of sextic binary forms via geometric invariant theory - [http://arxiv.org/abs/2002.09472v3] Geometric rank of tensors and subrank of matrix multiplication - [http://arxiv.org/abs/2511.11499v1] Faster MAX-CUT on Bounded Threshold Rank Graphs - [http://arxiv.org/abs/1208.5345v2] Generating All Minimal Edge Dominating Sets with Incremental-Polynomial Delay

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_max_cut_instance(n):
        variables = list(range(n))
        edges = [(i, j) for i in range(n) for j in range(i + 1, n)]
        cut_edges = random.sample(edges, n // 2)
        return variables, cut_edges

    def evaluate_polynomial(terms, x_values):
        result = Fraction(0)
        for term in terms:
            value = 1
            for var, exp in term:
                value *= x_values[var] ** exp
            result += term[2] * value
```

```
    return result

def construct_moment_matrix(polynomial, variables):
    n = len(variables)
    M = [[0] * (n + 1) for _ in range(n + 1)]
    for term in polynomial:
        x_values = [Fraction(1)] * (n + 1)
        for var, exp in term:
            x_values[var] = Fraction(term[2], 1) ** (exp / n)
        for i in range(n + 1):
            for j in range(n + 1):
                M[i][j] += x_values[i] * x_values[j]
    return M

def gaussian_elimination(matrix, b):
    n = len(matrix)
    augmented_matrix = [row + [b[i]] for i, row in enumerate(matrix)]
    for i in range(n):
        max_row = max(range(i, n), key=lambda r: abs(augmented_matrix[r][i]))
        augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_
        denom = augmented_matrix[i][i]
        if denom == 0:
            continue
        for j in range(n + 1):
            augmented_matrix[i][j] /= denom
        for k in range(n):
            if k != i and augmented_matrix[k][i] != 0:
                factor = augmented_matrix[k][i]
                for j in range(n + 1):
                    augmented_matrix[k][j] -= factor * augmented_matrix[i][j]
    return [row[-1] for row in augmented_matrix]

def rank(matrix):
    n = len(matrix)
    M = [row[:] for row in matrix]
    r = 0
    for i in range(n):
        if sum(M[j][i] ** 2 for j in range(i, n)) == 0:
            continue
        r += 1
        for j in range(n):
            M[i], M[j] = M[j], M[i]
            for k in range(n + 1):
                M[j][k] /= M[i][i]
            for l in range(n):
                if l != i:
                    factor = M[l][i]
                    for k in range(n + 1):
                        M[l][k] -= factor * M[i][k]
    return r

def approximation_ratio(polynomial, variables, cut_edges):
    n = len(variables)
    x_values = [Fraction(1)] * (n + 1)
    p_value = evaluate_polynomial(polynomial, x_values)
    for var in range(n):
```

```
            x_values[var] = Fraction(1 - 2 * random.randint(0, 1), 1)
        q_value = evaluate_polynomial(polynomial, x_values)
        return abs(q_value / p_value)

    n = random.choice([5, 10, 15, 20, 30, 40])
    variables, cut_edges = generate_max_cut_instance(n)
    d = len(cut_edges)
    polynomial = []
    for var in range(n):
        polynomial.append(((var,), 1, Fraction(1)))
    for i, j in cut_edges:
        polynomial.append(((i,), 1, Fraction(-1)))
        polynomial.append(((j,), 1, Fraction(-1)))

    M_p = construct_moment_matrix(polynomial, variables)
    rank_M_p = rank(M_p)

    if rank_M_p < d * math.log(n) ** 2:
        ratio = approximation_ratio(polynomial, variables, cut_edges)
        conjecture_holds = ratio > 0.878
        counterexample = "" if conjecture_holds else f"Approximation ratio {ratio} <= 0.878"
    else:
        conjecture_holds = True
        counterexample = ""

    return {
        "metric_name": "Rank of Moment Matrix",
        "metric_value": rank_M_p,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(re
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(i for i, res in enumerate(results) if not res["conjecture_holds'
        print(f"RESULT: FALSIFIED counterexample=\"Approximation ratio <= 0.878\" first_failing_se
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_84447bde.py", line 128, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_84447bde.py", line 104, in run_trial
    M_p = construct_moment_matrix(polynomial, variables)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_84447bde.py", line 41, in construct_mom
    for var, exp in term:
        ^^^^^^^^
ValueError: not enough values to unpack (expected 2, got 1)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means we cannot verify whether the conjecture holds or not. | next: Re-run the test with proper error handling to ensure it completes without crashing and produces results.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11245
2	propose	ollama_remote	glm4:latest	0	0	19247
3	propose	ollama_remote	glm4:latest	0	0	9947
4	preregistration	ollama_remote	glm4:latest	0	0	5840
5	novelty	ollama_remote	glm4:latest	0	0	4641
6	novelty	ollama_remote	glm4:latest	0	0	6478
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18215
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11742
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12241
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15690
11	judge	ollama_remote	glm4:latest	0	0	11079

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 126365 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/6ade8b9067eb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6ade8b9067eb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/6ade8b9067eb.tar.gz` (if generated)